DS Lab 4: Linear Regression

Boston Housing Dataset

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.datasets import fetch_openml
```

In [2]:
```
# Load Boston Housing Dataset
boston = fetch_openml(name='boston', version=1, as_frame=True)
df = boston.frame
df.head()
```

Out[2]:

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296.0	15.3	396.9
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242.0	17.8	396.9
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242.0	17.8	392.8
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222.0	18.7	394.6
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222.0	18.7	396.9

In [3]:
```
# Basic info
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   CRIM     506 non-null    float64
 1   ZN       506 non-null    float64
 2   INDUS    506 non-null    float64
 3   CHAS     506 non-null    category
 4   NOX      506 non-null    float64
 5   RM       506 non-null    float64
 6   AGE      506 non-null    float64
 7   DIS      506 non-null    float64
 8   RAD      506 non-null    category
 9   TAX      506 non-null    float64
 10  PTRATIO  506 non-null    float64
 11  B        506 non-null    float64
 12  LSTAT    506 non-null    float64
 13  MEDV     506 non-null    float64
dtypes: category(2), float64(12)
memory usage: 49.0 KB
```

Out[3]:

	CRIM	ZN	INDUS	NOX	RM	AGE	D
count	506.000000	506.000000	506.000000	506.000000	506.000000	506.000000	506.0000
mean	3.613524	11.363636	11.136779	0.554695	6.284634	68.574901	3.7950
std	8.601545	23.322453	6.860353	0.115878	0.702617	28.148861	2.1057
min	0.006320	0.000000	0.460000	0.385000	3.561000	2.900000	1.1296
25%	0.082045	0.000000	5.190000	0.449000	5.885500	45.025000	2.1001
50%	0.256510	0.000000	9.690000	0.538000	6.208500	77.500000	3.2074
75%	3.677083	12.500000	18.100000	0.624000	6.623500	94.075000	5.1884
max	88.976200	100.000000	27.740000	0.871000	8.780000	100.000000	12.1265

In [4]:
```
# Convert all columns to numeric
df = df.apply(pd.to_numeric, errors='coerce')
```

In [5]:
```
# Features and Target
X = df.drop('MEDV', axis=1)
y = df['MEDV']
```

In [6]:
```
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_
```

In [7]:
```
# Train Linear Regression model
model = LinearRegression()
model.fit(X_train, y_train)
```

Out[7]:

▾ LinearRegression

▶ Parameters

In [8]:
```
# Predictions
y_pred = model.predict(X_test)
```

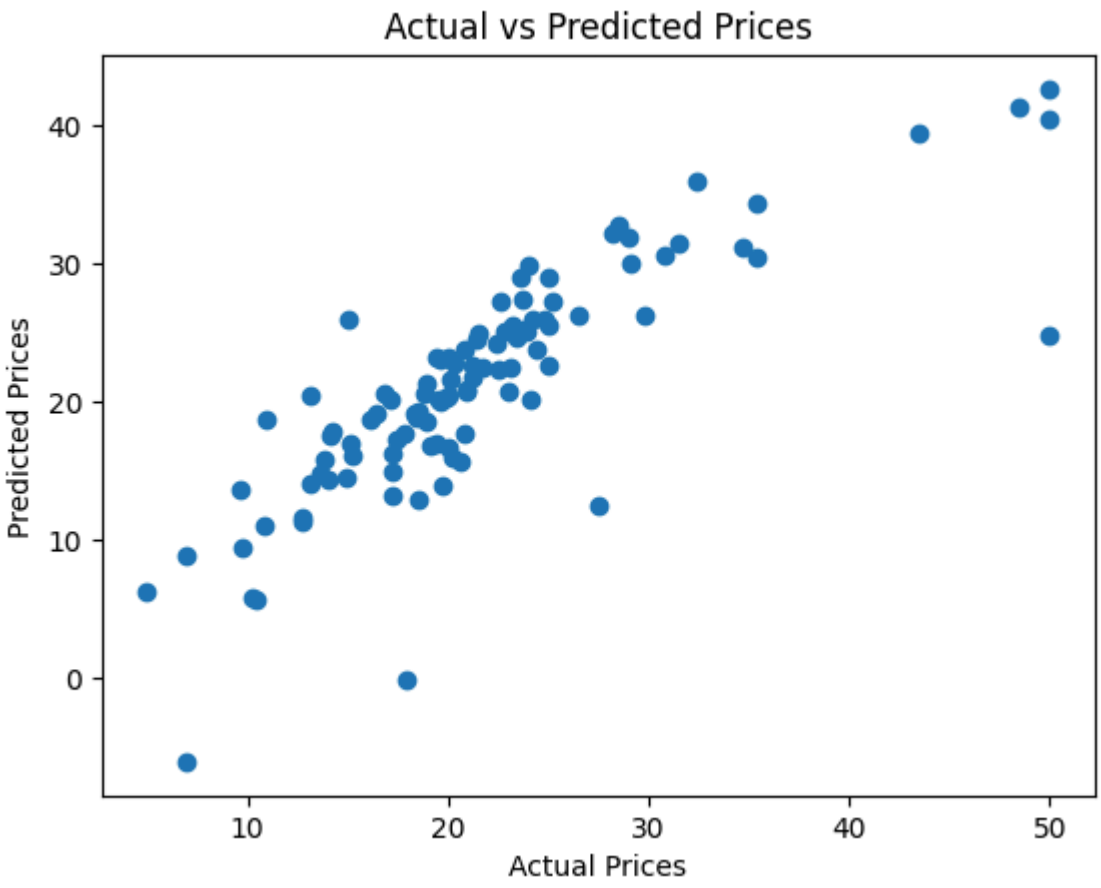
In [9]:
```
# Evaluation Metrics
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print("MSE:", mse)
print("RMSE:", rmse)
print("R2 Score:", r2)
```

```
MSE: 24.291119474973538
RMSE: 4.928602182665339
R2 Score: 0.6687594935356317
```

In [10]:
```
# Plot Actual vs Predicted
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
```

```
plt.title("Actual vs Predicted Prices")
plt.show()
```



In [11]:
```
# Model coefficients
coeff_df = pd.DataFrame(model.coef_, X.columns, columns=['Coefficient'])
coeff_df
```

Out[11]:

	Coefficient
CRIM	-0.113056
ZN	0.030110
INDUS	0.040381
CHAS	2.784438
NOX	-17.202633
RM	4.438835
AGE	-0.006296
DIS	-1.447865
RAD	0.262430
TAX	-0.010647
PTRATIO	-0.915456
B	0.012351
LSTAT	-0.508571